

Structural Properties of Standard Tries

1 Notation

Symbols

Let S be the set of stored strings. We use:

$$\begin{aligned} s &= |S|, \\ n &= \text{total length of all strings in } S, \\ d &= \text{alphabet size}, \\ T &= \text{the standard trie built from } S. \end{aligned}$$

The total input length is

$$n = \sum_{w \in S} |w|.$$

Example

For

$$S = \{\text{cat}, \text{cap}, \text{dog}\},$$

we have

$$s = 3$$

and

$$n = |\text{cat}| + |\text{cap}| + |\text{dog}| = 3 + 3 + 3 = 9.$$

2 Maximum Degree

Maximum Degree

Every internal node in a standard trie has at most

$$d$$

children.

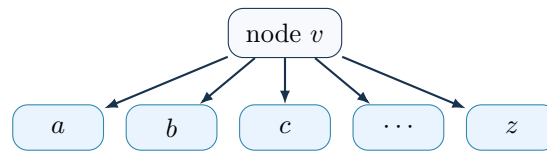
This is because each outgoing branch corresponds to one alphabet symbol. If the alphabet is

$$\Sigma = \{a, b, c, \dots, z\},$$

then

$$d = |\Sigma| = 26.$$

So an English-letter trie node can have at most 26 outgoing branches.



At most one child is needed for each possible next character.

3 External Nodes

External Node Count

If each string is represented by one terminal leaf, then a trie storing s strings has

s external nodes.

Each external node corresponds to exactly one stored string.

Example

For

$$S = \{\text{bear, bell, bid, bull, buy, sell, stock, stop}\},$$

there are

$$s = 8$$

stored strings, so the trie has

8 external nodes.

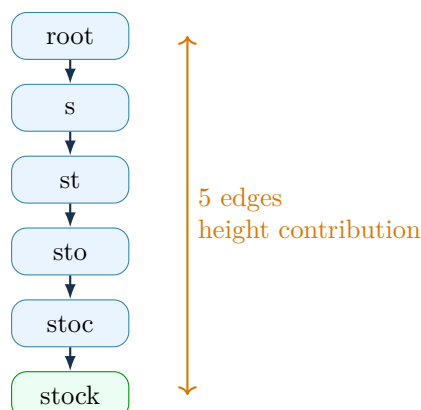
4 Height

Height of a Standard Trie

The height of a standard trie equals the length of the longest stored string:

$$h = \max_{w \in S} |w|.$$

A root-to-leaf path spells a complete stored string. Therefore, the longest root-to-leaf path corresponds to the longest word.



For the word set above, the longest word is

stock,

so

$$h = 5.$$

5 Number of Nodes

Node Bound

For a standard trie built from strings with total length n ,

$$|T| = \mathcal{O}(n).$$

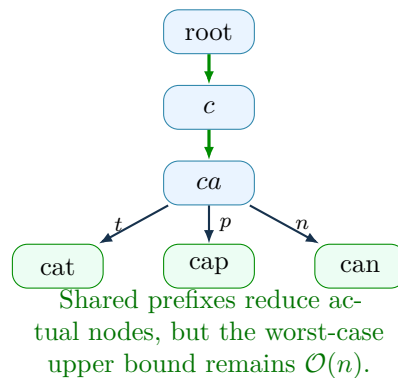
The reason is simple: while inserting all strings, each character can create at most one new trie node. Some characters create no new node because their prefix path already exists.

Prefix Sharing Example

For

cat, cap, can,

the prefix *ca* is shared.



Worst-Case Example

If stored strings share no useful prefix, little space saving occurs. For example,

cat, dog

separate immediately at the root because their first characters are different.

Even in this case, the number of nodes grows linearly with the total number of characters:

$$|\text{cat}| + |\text{dog}| = 6,$$

so the trie has only $\mathcal{O}(6)$ character-created nodes.

6 Search Complexity

Let m be the length of the word being searched. At each level, a simple implementation may check up to d child branches.

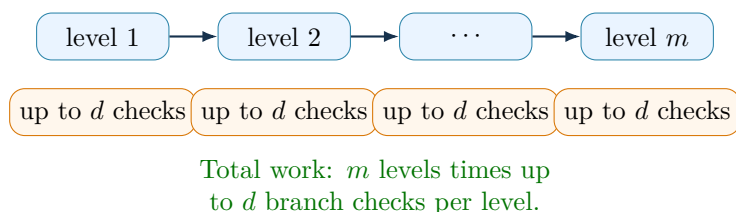
Search Bound

For a word of length m , the recorded search complexity is

$$\mathcal{O}(dm).$$

This means:

at most d child checks per level $\times m$ levels.



Implementation Note

If each node stores children in a direct array or hash table, child lookup can be faster than scanning all d children. The $\mathcal{O}(dm)$ bound corresponds to checking among up to d possible branches at each level.

Child representation	Search time for length m
Scan child list	$\mathcal{O}(dm)$
Sorted child list	$\mathcal{O}(m \log d)$
Hash table / direct array	expected $\mathcal{O}(m)$

7 Property Sheet

Property	Standard trie value
External nodes	s , one per stored string
Maximum degree	d , the alphabet size
Height	$\max_{w \in S} w $
Total nodes	$\mathcal{O}(n)$, where $n = \sum_{w \in S} w $
Search cost	$\mathcal{O}(dm)$ for query length m under branch scanning